

Benchmarking Generative Trajectory Models for Active-Inference Control

Yulin Li¹[0009-0001-0135-4332], Mohsen A. Jafari¹[0000-0003-4776-0632], and
Andrea Matta²[0000-0003-3902-2007]

¹ Rutgers University-New Brunswick, Piscataway NJ 08854, USA

² Politecnico di Milano, Via La Masa 1, 20156 Milano, Italy

Abstract. Learning from trajectory demonstrations offers a route to active-inference control of complex systems whose dynamics are difficult to model explicitly. We introduce generative active-inference control (GenAIF), in which one generative trajectory model learns from demonstrations and measured action interventions to supply a goal-conditioned policy distribution and a state-to-observation likelihood mapping. From this control design, we derive three model requirements: (i) useful action proposals, (ii) accurate prediction under imposed actions, and (iii) probabilistic observation evidence for belief updating and expected information gain. We benchmark diffusion, autoregressive Transformers, conditional variational autoencoders (CVAEs), and flow matching in a MuJoCo manipulation task with multiple physical conditions. Diffusion delivers the strongest control across the tested dynamics, while CVAE combines comparable short-horizon prediction with much faster inference. Correct conditioning is decisive, and trajectory reuse offers further computational savings. With the same frozen models, a hidden-dynamics experiment demonstrates prompt belief adaptation after an unannounced tilt change; subsequent instability identifies sustained inference as a remaining challenge. These findings support the use of shared generative trajectory models to connect action proposal, controlled prediction, and observation evidence within GenAIF.

Keywords: Active inference · Generative trajectory models · Capability benchmark · Belief adaptation

1 Introduction

People learn how to act by trying, observing, and adjusting. Watching someone learn a movement can help us adapt to the same unfamiliar physical dynamics [1]. Active Inference (AIF) offers an elegant account of how organisms bring perception and action together: an internal generative model connects hidden causes to sensory observations, and perception updates beliefs by minimizing variational free energy (VFE), an upper bound on sensory surprise [2,3]. Action selection looks ahead through expected free energy (EFE), combining preferences

Code: <https://github.com/lyeeonardo/generative-trajectory-benchmark>

for useful, safe outcomes with the information that an action is expected to reveal [5,6].

Our goal is a general framework that learns how a complex system behaves and how to control it from trajectory demonstrations. It should accommodate prior knowledge of system structure and hidden dynamics when available, and eventually learn their structure, form, and range when such knowledge is limited or absent.

Realizing this goal is computationally demanding: the controller must predict uncertain consequences while searching continuous action sequences [7,8]. Learned proposals can concentrate a finite EFE-evaluation budget on useful futures. We first explored this route in autonomous parking, combining diffusion-generated strategies with AIF evaluation and observation-driven adjustment [21]. Although the approach generalized to new parking settings, its design was tailored to that task. This experience led us to ask how much of the active-inference loop a single learned trajectory model could support.

Can a single learned generative trajectory model provide both a proposal distribution over policies and action-conditioned predictions of future observations sufficient for belief updating and expected-free-energy evaluation in active inference?

We propose generative active-inference control (GenAIF) around this shared model of action–observation trajectories. The same learned parameters support candidate generation and prediction under supplied actions; predictive evidence then supports belief updates about hidden operating conditions. The controller uses these operations to alternate between observing, updating its belief, and selecting actions according to task preferences and expected information gain. We learn the model offline from demonstrations supplemented with measured action interventions. At this stage, the hidden variables and their possible values are specified.

This design leaves a concrete model-selection problem. Which capabilities must a trajectory generator provide, and which generative families can provide them together at a useful computational cost? We identify three requirements: actionable trajectory proposals, accurate prediction of observations under externally imposed actions, and predictive evidence that can distinguish hidden conditions using actual observations. A plausible generated trajectory does not by itself establish the latter two capabilities. These requirements give us a way to test our framework’s learned component and offer other researchers practical criteria for combining AIF with generative trajectory models.

We design two groups of experiments in a custom MuJoCo ball-pushing environment to evaluate diffusion, autoregressive Transformers, conditional VAEs, and flow matching. The first benchmarks the required model capabilities and computational cost across different physical conditions. The second uses the same trained models to track a hidden dynamics change from replayed actions and observations, testing whether their predictive evidence remains reliable over time. At the tested settings, diffusion gives the strongest direct control, while CVAE provides similar short-horizon position prediction at lower cost. Reuse reduces

computation with model-dependent effects on control. All four models initially identify the dynamics change but fail to sustain confidence; belief-guided action selection remains open.

The contributions of this paper are as follows: (1) we propose GenAIF, in which a single model learned from trajectory demonstrations supplies a goal-conditioned policy distribution and a state-to-observation likelihood mapping; (2) we develop a MuJoCo environment and benchmark to evaluate the capabilities required of this shared model; and (3) we compare four generative model families in performance and computational cost, and demonstrate belief updating under changing hidden dynamics through a replay-based experiment.

2 GenAIF Framework

Learning a shared generative model.

Let s_t contain the physical variables and operating conditions needed to predict the system’s evolution. Some coordinates are observed, others hidden. The controller applies action a_t and receives observation o_{t+1} . We learn a conditional generative trajectory model offline from labeled demonstrations and measured action interventions. The learned parameters θ remain fixed during control.

For horizon H , write future actions as $U = a_{t:t+H-1}$, states as $S_+ = s_{t+1:t+H}$, and observations as $Y = o_{t+1:t+H}$. We represent candidate policies by action sequences U , replanned after each observation. With proposal context $c_k = (s_k, h_k, g)$, a causal factorization of the shared model is

$$p_\theta(U, S_+, Y \mid c_t) = \prod_{k=t}^{t+H-1} \mu_\theta(a_k \mid c_k) T_\theta(s_{k+1} \mid s_k, a_k) O_\theta(o_{k+1} \mid s_{k+1}, a_k). \quad (1)$$

Here $h_t = (o_0, a_0, \dots, a_{t-1}, o_t)$ is the available history, extended by generated actions and observations at future steps. The factors describe action generation (μ_θ), state transitions (T_θ), and observations (O_θ); they can share learned parameters. The task specification g guides proposals and preferences, while physical predictions depend on the system and imposed actions.

For interacting subsystems, let $s_t = (s_t^1, \dots, s_t^m)$. A known, fixed graph $\mathcal{G}$ specifies the parents of each next-state component. Under conditional independence given these parents and the action,

$$T_\theta(s' \mid s, a, \mathcal{G}) = \prod_{i=1}^m T_{\theta,i}(s'^i \mid s^{\text{Pa}_{\mathcal{G}}(i)}, a). \quad (2)$$

Parents can include the component itself. This allows hidden conditions to persist, drift, or switch within a fixed transition model, provided those transitions are represented in data or prior knowledge. Graph structure is optional; we suppress $\mathcal{G}$ in the remaining notation.

The shared model must therefore support two queries: predict the next state from the current state and an imposed action, and generate an H -step trajectory

from the current state, history, and task condition g :

$$\begin{aligned} s_{t+1} &\sim T_\theta(\cdot \mid s_t, a_t), \\ (U, S_+, Y) &\sim p_\theta(\cdot \mid s_t, h_t, g). \end{aligned} \quad (3)$$

For this benchmark, measured physical variables are fully observed, and the representation and range of hidden dynamics are known. The first experiment group supplies tilt; the belief-tracking experiment infers lateral tilt among three known values using a prescribed context-transition matrix. The following equations describe the general inference-and-control loop; the benchmark specializes this formulation to inference over hidden tilt.

Within each model family, proposal and prediction reuse the same trained trajectory backbone or decoder, operating on alternating action and observation tokens. Queries differ in role, known-coordinate mask, and conditioning. Proposal queries generate future actions and observations using task and quality conditions. Prediction queries fix the supplied actions, remove goal and quality conditioning, and generate observations under the physical context. Calibrated predictive samples supply approximate observation likelihoods without a separately trained dynamics model. In AIF terms, these operations provide a goal-conditioned policy distribution and an approximate state-to-observation likelihood mapping, conditioned on history and imposed actions.

Inferring the current state.

The controller maintains a belief $b_t(s) \approx p_\theta(s_t = s \mid h_t)$ over the current state [4]. After executing a_t , it predicts how that state evolves and corrects the prediction using the actual observation:

$$\begin{aligned} b_{t+1}^-(s') &= \int T_\theta(s' \mid s, a_t) b_t(s) ds, \\ b_{t+1}(s') &\propto O_\theta(o_{t+1} \mid s', a_t) b_{t+1}^-(s'). \end{aligned} \quad (4)$$

This correction minimizes variational free energy over normalized beliefs q :

$$F_{t+1}[q] = D_{\text{KL}}(q \parallel b_{t+1}^-) - \mathbb{E}_q[\log O_\theta(o_{t+1} \mid s', a_t)]. \quad (5)$$

The first term retains information from the predicted belief; the second rewards explaining the observation [3]. An unrestricted q recovers the Bayesian posterior; a restricted family approximates it. Online adaptation therefore changes the state belief while keeping the learned model fixed.

Proposing and selecting actions.

Given b_t , we sample state hypotheses $s^{(j)} \sim b_t$, generate joint trajectories using Eq. (1), and retain their action sequences $U^{(j)}$. To evaluate a candidate, we impose its actions on the model and predict across the same full belief:

$$Q_t^U(\mathbf{S}, Y) = b_t(s_t) \prod_{k=t}^{t+H-1} T_\theta(s_{k+1} \mid s_k, a_k) O_\theta(o_{k+1} \mid s_{k+1}, a_k), \quad (6)$$

where $\mathbf{S} = s_{t:t+H}$. Removing the action-generation factors μ_θ represents the intervention $\text{do}(U)$. Every candidate is evaluated under the common belief, including hypotheses other than the one that generated it.

We use the following cost-augmented approximation to EFE for action selection [5,6]:

$$G_t(U) = \underbrace{\mathbb{E}_{Q_t^U}[C(U, Y; g)]}_{\text{expected task cost}} - \lambda \underbrace{I_{Q_t^U}(\Xi; Y)}_{\text{expected information gain}}, \quad \lambda \geq 0. \quad (7)$$

Here C specifies task and action costs, $\Xi = \chi(\mathbf{S})$ is the hidden state quantity of interest, and λ sets the value of information in cost units. Mutual information measures how much future observations are expected to reduce uncertainty about Ξ . Its predictive likelihood must also support the update from actual observations.

Among candidates satisfying action bounds and a predicted-risk limit, select the one with minimum G_t , execute its first action, observe, and update the belief. This closes the loop: the belief shapes proposals and their evaluation, while the selected action produces the evidence for the next belief. All hypothetical outcomes come from the learned model.

Shared masked queries and calibrated predictive likelihoods approximate the framework’s distributions in the benchmark. The belief-tracking experiment evaluates updates on replayed actions; the full inference-and-action loop remains the broader framework.

3 Required Capabilities and Candidate Models

Our finite-candidate GenAIF controller imposes three evaluation requirements. The two queries in Eq. (3) supply candidates and their predicted consequences, but using predictions for belief updating and expected information gain also requires reliable observation evidence. We therefore assess proposal usefulness, controlled prediction, and evidence quality separately; the latter two are different demands on the same predictive model.

Action proposals. With a finite action-search budget, the proposal distribution determines which action sequences are evaluated using G . Even perfect scoring cannot select an action that was never proposed. We therefore require useful, executable action sequences.

Controlled prediction. A proposal generated assuming a flat board must also be evaluated under tilted-board hypotheses in the current belief. This requires predicting the consequences of the same imposed actions across that belief, as in Eq. (6). Observations sampled alongside the original proposal represent only one possible future. Controlled predictions supply the outcome distribution needed to evaluate task cost, safety, and possible future observations.

Observation evidence. A forecast must also say how strongly an observed outcome supports each hypothesis. Predictors with identical means can produce very different belief updates if their uncertainty differs. The predictive law must

provide normalized likelihoods for the VFE update after execution and expected information gain in G before execution [4]. Forecast accuracy and evidence quality therefore need separate evaluation. If hypotheses predict indistinguishable observations, the observation should not change their relative belief.

These requirements motivate four candidate families whose literature supports relevant proposal, prediction, or density-evaluation operations.

Diffusion. Diffusion models [13,17] offer precedents for trajectory planning in Diffuser and Decision Diffuser, and action generation in Diffusion Policy [16,22,15]. CSDI demonstrates completion from observed time-series coordinates [23], motivating conditional prediction; score-based formulations also offer a route to likelihood evaluation [14].

Autoregressive Transformer. Transformers model sequential dependencies [20]. Trajectory Transformer demonstrates joint state-action modeling and planning [18], while Decision Transformer supports return-conditioned action generation [24]. With probabilistic outputs, an autoregressive factorization offers both forecasts conditioned on supplied actions and evaluable conditional densities, motivating prediction and observation evidence alongside proposals.

Conditional variational autoencoder. Conditional VAEs learn context-dependent distributions through latent variables [25,12]. Play-LMP provides evidence for goal-directed action generation [26], and Trajectron++ for probabilistic forecasts conditioned on an ego-agent’s planned motion [27]. These support the proposal and prediction hypotheses; using such predictions as observation evidence additionally requires accounting for latent-variable uncertainty.

Flow matching. Flow matching provides conditional sampling and, through its continuous-flow formulation, a route to density evaluation [19]. Trajectory Conditional Flow Matching demonstrates forecasting and goal-conditioned planning [28], while π_0 generates robot action sequences [29]. These precedents motivate both prediction and proposal use.

The cited studies motivate the candidates without establishing all three capabilities in one checkpoint. Conditional completion alone does not establish accurate prediction under imposed actions or reliable observation evidence. Our benchmark tests shared proposal and prediction interfaces, then evaluates their evidence for inference, alongside supplied-condition sensitivity, trajectory reuse, and computation.

4 Experimental Setup

Task. Our custom MuJoCo environment [30] requires a rectangular pusher to move a ball uphill, around an obstacle, and settle it safely in a goal region (Fig. 1). We vary lateral tilt across $\{-15, 0, +15\}^\circ$ and use three obstacle positions, giving nine physical conditions. Longitudinal tilt stays at $+20^\circ$. This tests whether one model can adjust its control to different dynamics while solving the same task. An episode ends on success, ball-obstacle collision, falling off the board, or the control-step limit.

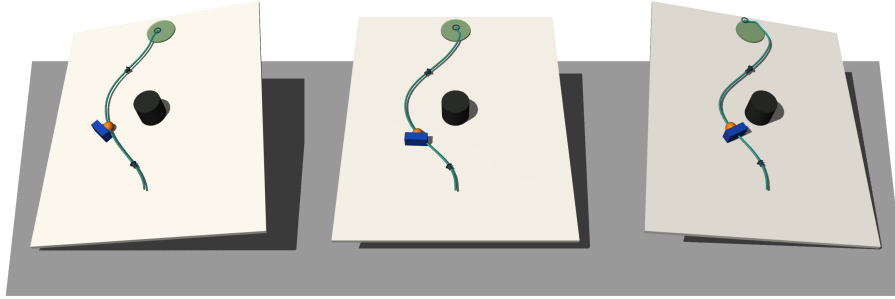


Fig. 1. Ball-pushing task at illustrative lateral tilts. The pusher moves the ball around the obstacle toward the green goal; lines illustrate routes.

Actions specify planar pusher velocity and yaw rate. The observation records ball position and velocity, pusher position, and pusher yaw. Each query receives five observations and four preceding actions, together with geometry, supplied tilt, and elapsed time; prediction horizons are $H \in \{1, 3, 6\}$ steps. The simulator is deterministic, so repeated trials measure variation in generative sampling.

Dataset. A feedback controller supplies expert demonstrations in every physical condition. From 216 successful reference episodes, we collect 65,195 short action interventions by restoring recorded simulator states and executing reference variants, perturbations, and exploratory actions. Demonstrations supply useful behaviour, while these interventions reveal what happens when actions depart from it. Prediction training retains unsafe outcomes; proposal training uses references and safe intentional continuations. Windows are truncated at physical termination, and padding is excluded from the loss.

Training, development, calibration, and test sets are separated by source rollout: each intervention stays with its reference episode. We remove exact duplicates and compute normalization from training data only. All sets cover the same physical conditions, so evaluation asks whether the models can handle new rollouts within this task. Private simulator state is never an input.

Training and selection. All models complete the same training-data exposure budget. We select checkpoints on development performance in proposal usefulness, controlled prediction, and observation likelihood. Each selected checkpoint serves both proposal and prediction queries and remains fixed during evaluation. Test data influence neither selection nor calibration.

Model sizes share a fixed maximum parameter budget. We retain the smallest tested model when additional capacity no longer improves development performance. Diffusion continued to benefit from scaling, motivating its larger selected configuration. Training recipes also differ: diffusion’s task-directed proposal and completion examples come from successful references, while the other families additionally use safe intervention continuations. These differences, together with model-specific losses and samplers, make this a comparison of the tested implementations.

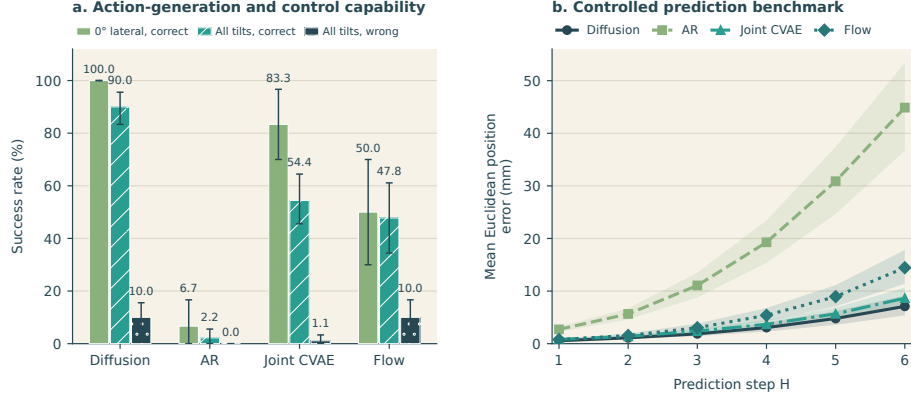


Fig. 2. E1 control and prediction. (a) Control success with correct or incorrect tilt inputs (zero-lateral-tilt subset: 30 episodes; all tilts: 90 per group). (b) Mean Euclidean ball-position error of the predictive mean over six steps. Whiskers and shading show 95% bootstrap intervals over 10 sampling seeds (a) and 18 source trajectories (b); bands are pointwise. AR: autoregressive Transformer.

For belief updating, we construct Gaussian observation likelihoods from each model’s predictive samples, treat yaw as circular, and calibrate uncertainty on a separate data split. This supplies a common way to turn forecasts into evidence about hidden conditions. We measure model evaluation time with synchronization on an NVIDIA RTX 5090. Detailed architectures, optimization settings, and sampling configurations are provided in the code.

5 Experiments and Results

We evaluate the shared model in two experiment groups: the capability benchmark (E1) and the belief-tracking experiment (E2). In E1, direct execution tests proposal usefulness, imposed-action queries test prediction, and single-step condition identification tests observation evidence. We also evaluate condition sensitivity, trajectory reuse, and computation. E2 uses the same frozen checkpoints and calibration to track a hidden tilt change, with every model receiving the same executed actions and observations for each trial.

Action generation across dynamics. We first test whether one learned model can control all nine physical conditions, using ten sampling seeds per condition for 90 episodes per setting. At each decision, it proposes one six-step trajectory, executes the first action, and replans. Tilt stays fixed within an episode. We compare correct and incorrect tilt inputs on matched scenes and seeds. This arm uses no online belief update or EFE-based candidate selection, so its success rate measures proposal usefulness under supplied conditions.

Diffusion provides the most reliable control across the tested dynamics: 90% success, compared with 54% for CVAE and 48% for flow (Fig. 2a). CVAE performs

Table 1. E1 control, prediction, and computation. Position RMSE uses 841 valid endpoints. Times cover model evaluation and checking per executed action, excluding simulator execution.

Model	Success /90		$H = 6$ position RMSE (mm)	Time/action (ms)	
	Direct	Reuse		Direct	Reuse
Diffusion	81	68	14.295	57.123	9.794
Autoregressive	2	2	65.809	48.735	12.982
CVAE	49	61	15.264	5.643	1.236
Flow matching	43	34	24.776	199.571	37.037

well in the zero-lateral-tilt cases but loses reliability across the broader set. Since the true tilt is supplied, these differences persist even without condition-estimation error. Accurate beliefs therefore need to be paired with a model that can turn the inferred condition into effective actions.

Conditioning is decisive: incorrect tilt inputs reduce diffusion’s success by 80 percentage points, with large losses also seen for CVAE and flow. A capable generator can therefore choose ineffective actions when conditioned on the wrong dynamics, making accurate hidden-condition inference essential to the eventual controller.

The autoregressive model succeeds in only 2/90 episodes and remains unreliable even in the zero-lateral-tilt subset. Interference between conditions during joint training is one hypothesis worth testing against single-condition training; this comparison does not measure loss of previously learned behaviour.

Controlled prediction. We impose the same action sequences at sampled test states and compare predicted ball motion with simulator outcomes. Queries retain physical context but omit the desired goal and quality conditioning. This asks whether the model predicts what supplied actions actually do, independently of the actions it would propose. Figure 2b follows position error across the prediction horizon.

Diffusion and CVAE produce the closest forecasts. At the sixth prediction step, mean Euclidean position error is 7.1 mm for diffusion and 8.7 mm for CVAE, compared with 14.4 mm for flow and 44.9 mm for the autoregressive model. Autoregressive error accumulates most rapidly, limiting extended action evaluation. CVAE’s accuracy remains close to diffusion’s despite weaker control: predicting consequences and generating useful actions are separate achievements.

Figure 2b reports mean Euclidean position error; Table 1 reports endpoint RMSE.

Computation and trajectory reuse. We compare generating a new trajectory at every decision with continuing the current trajectory while its predicted observations agree with actual motion. Both settings use the same physical cases and sampling seeds. Table 1 brings control success and endpoint prediction error together with model-plus-checker time per executed action. Timings include proposal re-prediction and exclude simulator execution.

Table 2. E1 hidden-tilt identification over 864 action–observation pairs. Lower log loss and calibration error indicate better probability estimates.

Model	Accuracy (%)	Log loss (nats)	Calibration error
Diffusion	98.96	0.540	0.062
Autoregressive	79.28	0.959	0.127
CVAE	99.07	0.434	0.057
Flow matching	99.07	0.474	0.069

CVAE has the lowest model-plus-checker time per executed action: 5.6 ms, compared with 57.1 ms for diffusion. Its accurate short-horizon predictions leave computational headroom for higher-frequency control. The choice is therefore between CVAE’s speed and diffusion’s stronger control across varying dynamics. Flow’s tested solver makes it the most expensive option.

Reuse reduces model-plus-checker time per action by 73–83%, but its value depends on the trajectory being followed. CVAE’s success rises by 13 percentage points, while diffusion’s falls by 14. Prediction agreement is a useful trigger for saving computation, but cannot by itself determine how long a plan should be followed. Reuse improves both speed and success for CVAE, while diffusion achieves higher control success with frequent regeneration. This suggests choosing the execution schedule jointly with the proposal model.

Evidence for hidden-condition inference. Starting from equal belief in the three tilt hypotheses, each model predicts the same imposed action under each hypothesis and uses the observed transition to update their probabilities. This tests whether forecasts supply useful evidence about hidden dynamics, beyond the accuracy of their means. Table 2 reports identification accuracy, log loss of the true-tilt probability, and calibration error comparing confidence with accuracy across ten bins.

Across 864 action–observation pairs, diffusion, CVAE, and flow achieve approximately 99% condition-identification accuracy, compared with 79% for the autoregressive model. CVAE also records the lowest log loss and calibration error, showing that probability quality differs even when identification accuracy is similar. These predictions provide the observation evidence required by GenAIF. E2 asks whether that evidence remains reliable when accumulated over a changing trajectory.

E2: Belief adaptation and evidence reliability. In twelve trials, we switch lateral tilt from 0° to $+15^\circ$ after ten actions; each has a matched trial that stays flat. All models receive identical replayed actions and measured observations; belief updates do not alter execution. Starting from a uniform belief, each model predicts the next observation under the three tilt hypotheses and applies

$$b_{t+1}(z') \propto \ell_\theta(o_{t+1} \mid h_t, a_t, z') \sum_z M_{z',z} b_t(z), \quad (8)$$

where z' governs the observed transition. The context-transition matrix M is fixed from development data and allows changes between hypotheses. The filter

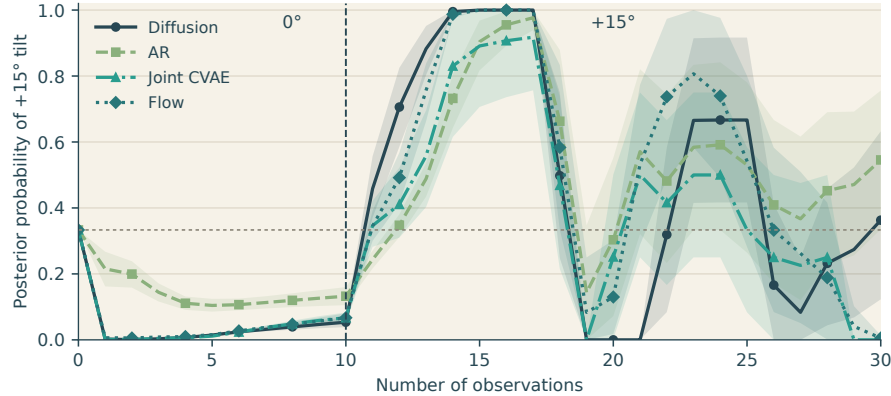


Fig. 3. E2 belief tracking after a $0^\circ \rightarrow +15^\circ$ tilt switch. Lines show mean $p(+15^\circ)$ across 12 trials; observation 11 first reflects the switch. Shading shows pointwise 95% bootstrap intervals over source episodes. The horizontal line marks the uniform prior.

receives neither a switch indicator nor its timing, and uses no posterior reset or test-time refitting.

Figure 3 shows prompt belief adaptation. During the initial flat phase, the probability assigned to $+15^\circ$ quickly falls. Matched flat trials remain correctly classified over observations 10–30. After the switch, all four models redirect belief toward $+15^\circ$: median delay to 90% posterior confidence is three observations for diffusion and four or five for the other models (Table 3). Both tilt values are represented in training, but switches within a trajectory are not. The initial adaptation shows that a frozen model can recognize a changed operating condition from observations, without demonstrations of that transition.

Belief later oscillates despite the unchanged tilt: no model maintains 90% confidence throughout observations 16–30 (0/12 trials per model). Every trial continues to supply observations, so this decline reflects inconsistent inference rather than missing data. Together with E1, this shows that accurate single-step identification and rapid initial adaptation do not guarantee stable sequential belief tracking. Observation evidence must therefore be evaluated over complete trajectories as well as individual transitions.

Table 3. E2 adaptation across 12 switched trials. Delay counts observations after the switch until $p(+15^\circ)$ first reaches 0.9; final correctness uses the most likely hypothesis at observation 30.

Model	Identification delay median [min,max]	Identified by six	Correct at observation 30
Diffusion	3 [2,4]	12/12	5/12
Autoregressive	5 [5,8]	10/12	7/12
CVAE	4 [3,10]	10/12	0/12
Flow matching	4 [3,4]	12/12	0/12

The pusher continues executing the original flat-board action sequence after the switch. Belief tracking is therefore tested while actions remain tailored to the previous dynamics, making prediction reliability along the resulting motion particularly important. Training trajectories keep their physical condition fixed and contain no explicit demonstrations of recovery after a dynamics switch. Missing recovery experience is therefore a plausible contributor to the unstable predictions. Its effect still needs to be separated from likelihood calibration and the prescribed transition prior. Demonstrations that include dynamics changes and corrective actions offer a concrete next step toward sustained adaptation within GenAIF.

6 Discussion

Our vision for GenAIF is to learn a system’s behaviour from rich trajectory demonstrations and use that understanding to connect inference with action. This paper takes a first step toward that vision: it derives the required model capabilities and benchmarks candidate families against them.

The central conclusion is that one shared trajectory model can provide a goal-conditioned policy distribution and an approximate state-to-observation likelihood mapping, supporting action proposals, controlled prediction, and initial belief revision. These capabilities must be evaluated together: plausible trajectories and accurate forecasts alone do not establish effective control. Diffusion offers more reliable proposals in the tested setting, while CVAE provides comparable short-horizon prediction at substantially lower query cost. Their suitability for GenAIF depends on the balance between proposal quality, prediction, evidence reliability, and computation. Accurate conditioning is fundamental to using either model effectively. The earlier proposal-only experiments in Appendix A provide complementary evidence for the action-search component.

The next step is to demonstrate sustained inference and the complete belief-guided control loop across a wider range of systems. We will test whether richer recovery demonstrations and better calibrated observation evidence stabilize tracking, and whether expected information gain changes actions in ways that improve control. This extends the current benchmark from component capabilities to their interaction during execution.

Looking ahead, we want GenAIF to control complex industrial systems whose interacting dynamics make detailed models impractical to construct or maintain with traditional methods. Rich trajectory demonstrations would supply the main learning signal, complemented by limited prior knowledge of system structure and the form and range of hidden dynamics. A graph would encode subsystem relationships and organize learned transition models into a model of the whole system. Our aim is active-inference control that infers hidden operating conditions, anticipates consequences, and pursues specified goals while seeking information when needed. Learning more of the graph and hidden-dynamics representation from data would progressively reduce the prior knowledge required.

A Proposal Models with Simulator-Based Planning

The main paper evaluates learned models as world models. This appendix instead tests proposal distributions with MuJoCo transitions, measuring candidate quality and finite-budget control when an accurate simulator is available. It provides practical guidance for model-based planners, but does not demonstrate learned dynamics or the complete GenAIF loop.

All controllers use simulator rollouts, common preference and safety scoring, an eight-step horizon, and receding-horizon execution. The learned proposals come from a separate archive of successful RL-specialist demonstrations; their data and checkpoints differ from E1 and E2. We compare them with the cross-entropy method (CEM) [9,10]. Six scenes and three sampling seeds give eighteen episodes per family and budget. “Proposal model” pools five separately evaluated controllers with equal family weight: BC-MDN [11], CVAE, diffusion, flow, and autoregressive. Gated controllers mix learned and fallback candidates.

Learned proposals improve control with limited search. We vary the candidate budget while keeping rollout and scoring shared. At just eight candidates, learned proposals succeed in most trials, while CEM fails throughout (Fig. A1A). Gating raises reliability further and solves nearly every trial at the largest common budget. The improvement comes from supplying useful alternatives to the planner: a good transition model alone cannot compensate for a poor candidate set.

More extensive CEM search closes much of the success gap at greater planning cost (Fig. A1B). Because this configuration also doubles refinement iterations and changes the elite rule, it represents a stronger search rather than a candidate-count change alone. Learned proposals use small search budgets effectively; extensive simulator search remains a viable alternative.

Diffusion favours candidate quality; CVAE favours speed. To separate proposal quality from candidate count, we compare all five families at $K = 128$ (Table A1). Diffusion has the highest fraction of feasible candidates and succeeds in every trial. CVAE nearly matches its success with much cheaper generation. The gap in total planning time is much smaller because simulator rollouts and scoring dominate the fast generators. Model selection should therefore use complete planning cost as well as proposal latency.

The advantage survives comparable computation and new layouts. We calibrate candidate counts to CPU planning targets and repeat the evaluation

Table A1. Proposal-family comparison at $K = 128$. Feasibility averages the fraction of feasible candidates per rollout; times are per decision.

Family	Success	Feasible (%)	Proposal (ms)	Plan (ms)
BC-MDN	11/18	82.4	0.75	346
CVAE	17/18	92.1	0.39	352
Diffusion	18/18	100.0	50.05	411
Flow	14/18	83.9	3.86	354
Autoregressive	14/18	82.1	14.03	360

Table A2. CEM and CVAE at a 50 ms CPU target. Transfer uses new start-goal scenes; planning time includes proposal generation, rollout, and scoring.

Method	K	Success /18		Plan time (ms)	
		Nominal	Transfer	Nominal	Transfer
CEM	8	0	0	44.5	44.4
CVAE	16	16	18	43.8	43.5

on six new start-goal scenes with the same three sampling seeds. On one Torch thread, we select the largest tested K whose 90th-percentile planning time fits the target, or the smallest K if none fits; offline training is excluded.

At the 50 ms target, CVAE and CEM have nearly identical measured planning cost (Table A2), yet CVAE succeeds in most nominal trials and every transfer trial while CEM solves neither. The proposal advantage therefore persists under comparable computation and new layouts; CVAE also leads the wider low-latency sweep (Fig. A1C,D).

The CVAE-CEM result uses measured timings; all findings use fixed checkpoints and small scene sets.

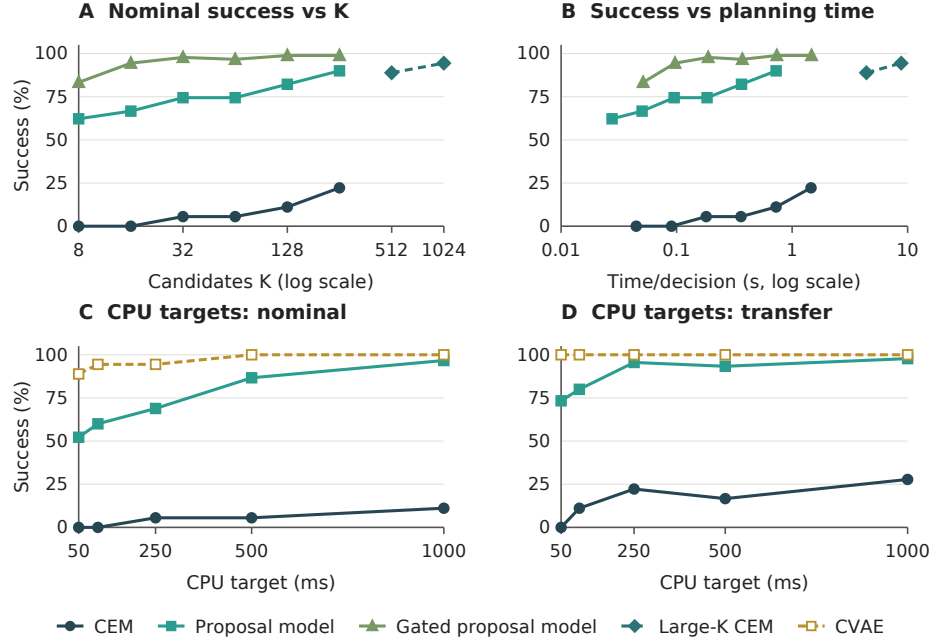


Fig. A1. Proposal-only control with MuJoCo transitions. Success versus (A) candidate budget, (B) measured planning time, and (C,D) requested CPU targets for nominal and new start-goal scenes. Diffusion exceeds the two smallest CPU targets, so pooled comparisons there are not at equal measured cost.

References

1. Mattar, A.A.G., Gribble, P.L.: Motor learning by observing. *Neuron* **46**(1), 153–160 (2005). doi:10.1016/j.neuron.2005.02.009
2. Friston, K.: The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience* **11**(2), 127–138 (2010). doi:10.1038/nrn2787
3. Friston, K., FitzGerald, T., Rigoli, F., Schwartenbeck, P., Pezzulo, G.: Active inference: a process theory. *Neural Computation* **29**(1), 1–49 (2017). doi:10.1162/NECO_a_00912
4. Da Costa, L., Parr, T., Sajid, N., Veselic, S., Neacsu, V., Friston, K.: Active inference on discrete state-spaces: a synthesis. *Journal of Mathematical Psychology* **99**, 102447 (2020). doi:10.1016/j.jmp.2020.102447
5. Parr, T., Pezzulo, G., Friston, K.J.: *Active Inference: The Free Energy Principle in Mind, Brain, and Behavior*. MIT Press, Cambridge, MA (2022). doi:10.7551/mitpress/12441.001.0001
6. Friston, K., Rigoli, F., Ognibene, D., Mathys, C., FitzGerald, T., Pezzulo, G.: Active inference and epistemic value. *Cognitive Neuroscience* **6**(4), 187–214 (2015). doi:10.1080/17588928.2015.1020053
7. Tschantz, A., Baltieri, M., Seth, A.K., Buckley, C.L.: Scaling active inference. In: *2020 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8. IEEE (2020). doi:10.1109/IJCNN48605.2020.9207382
8. Fountas, Z., Sajid, N., Mediano, P.A.M., Friston, K.: Deep active inference agents using Monte-Carlo methods. In: *Advances in Neural Information Processing Systems*, vol. 33, pp. 11662–11675. Curran Associates (2020).
9. Rubinstein, R.Y.: The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability* **1**, 127–190 (1999). doi:10.1023/A:1010091220143
10. Botev, Z.I., Kroese, D.P., Rubinstein, R.Y., L’Ecuyer, P.: The cross-entropy method for optimization. In: Govindaraju, V., Rao, C.R. (eds.) *Handbook of Statistics: Machine Learning: Theory and Applications*, vol. 31, pp. 35–59. Elsevier (2013). doi:10.1016/B978-0-444-53859-8.00003-5
11. Bishop, C.M.: Mixture density networks. Technical Report NCRG/94/004, Neural Computing Research Group, Aston University (1994)
12. Sohn, K., Lee, H., Yan, X.: Learning structured output representation using deep conditional generative models. In: Cortes, C., Lawrence, N., Lee, D., Sugiyama, M., Garnett, R. (eds.) *Advances in Neural Information Processing Systems*, vol. 28, pp. 3483–3491. Curran Associates (2015)
13. Ho, J., Jain, A., Abbeel, P.: Denoising diffusion probabilistic models. In: *Advances in Neural Information Processing Systems*, vol. 33, pp. 6840–6851. Curran Associates (2020)
14. Song, Y., Sohl-Dickstein, J., Kingma, D.P., Kumar, A., Ermon, S., Poole, B.: Score-based generative modeling through stochastic differential equations. In: *International Conference on Learning Representations* (2021)
15. Chi, C., Feng, S., Du, Y., Xu, Z., Cousineau, E., Burchfiel, B.C.M., Song, S.: Diffusion policy: visuomotor policy learning via action diffusion. In: *Robotics: Science and Systems XIX* (2023). doi:10.15607/RSS.2023.XIX.026
16. Janner, M., Du, Y., Tenenbaum, J.B., Levine, S.: Planning with diffusion for flexible behavior synthesis. In: *Proceedings of the 39th International Conference on Machine Learning*, Proceedings of Machine Learning Research, vol. 162, pp. 9902–9915 (2022).

17. Peebles, W., Xie, S.: Scalable diffusion models with transformers. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 4195–4205 (2023).
18. Janner, M., Li, Q., Levine, S.: Offline reinforcement learning as one big sequence modeling problem. In: *Advances in Neural Information Processing Systems*, vol. 34, pp. 1273–1286 (2021).
19. Lipman, Y., Chen, R.T.Q., Ben-Hamu, H., Nickel, M., Le, M.: Flow matching for generative modeling. In: *International Conference on Learning Representations* (2023)
20. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008. Curran Associates (2017)
21. Huang, Y., Li, Y., Matta, A., Jafari, M.A.: Navigating uncertainties with active inference and probabilistic diffusion. In: Albarracin, M., et al. (eds.) *Active Inference. IJWAI 2025*. Communications in Computer and Information Science, vol. 2857, pp. 416–429. Springer, Cham (2026). doi:10.1007/978-3-032-16955-6_24
22. Ajay, A., Du, Y., Gupta, A., Tenenbaum, J.B., Jaakkola, T., Agrawal, P.: Is conditional generative modeling all you need for decision-making? In: *International Conference on Learning Representations* (2023).
23. Tashiro, Y., Song, J., Song, Y., Ermon, S.: CSDI: conditional score-based diffusion models for probabilistic time series imputation. In: *Advances in Neural Information Processing Systems*, vol. 34, pp. 24804–24816 (2021).
24. Chen, L., Lu, K., Rajeswaran, A., Lee, K., Grover, A., Laskin, M., Abbeel, P., Srinivas, A., Mordatch, I.: Decision Transformer: reinforcement learning via sequence modeling. In: *Advances in Neural Information Processing Systems*, vol. 34, pp. 15084–15097 (2021).
25. Kingma, D.P., Welling, M.: Auto-encoding variational Bayes. In: *International Conference on Learning Representations* (2014).
26. Lynch, C., Khansari, M., Xiao, T., Kumar, V., Tompson, J., Levine, S., Sermanet, P.: Learning latent plans from play. In: *Proceedings of the Conference on Robot Learning*, Proceedings of Machine Learning Research, vol. 100, pp. 1113–1132 (2020).
27. Salzmann, T., Ivanovic, B., Chakravarty, P., Pavone, M.: Trajectron++: dynamically-feasible trajectory forecasting with heterogeneous data. In: *Computer Vision – ECCV 2020*, Part XVIII, Lecture Notes in Computer Science, vol. 12363, pp. 683–700. Springer, Cham (2020). doi:10.1007/978-3-030-58523-5_40
28. Ye, S., Gombolay, M.: Efficient trajectory forecasting and generation with conditional flow matching. In: *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2816–2823. IEEE (2024). doi:10.1109/IROS58592.2024.10802208
29. Black, K., Brown, N., Driess, D., et al.: π_0 : a vision-language-action flow model for general robot control. In: *Robotics: Science and Systems XXI* (2025). doi:10.15607/RSS.2025.XXI.010
30. Todorov, E., Erez, T., Tassa, Y.: MuJoCo: a physics engine for model-based control. In: *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033 (2012). doi:10.1109/IROS.2012.6386109